# WPT Day 4

**Sat 24 Oct 26** · Nullish coalescing ( `??` ), arrays & 2D arrays, JSON (nested + arrays of objects), optional chaining ( `?.` ), destructuring & spread, JS classes / inheritance / polymorphism, callbacks, arrow functions, `forEach` / `filter` / `map`, and async with `setTimeout`

---

## Table of Contents


1. Nullish Coalescing Operator ( `??` )
2. Arrays in JavaScript
3. 2D Arrays
4. JSON (JavaScript Object Notation)
5. Optional Chaining ( `?.` ) and Nested JSON Safety
6. Destructuring and the Spread Operator
7. OOP in JavaScript: Classes, Constructors, and Inheritance
8. Callback Functions, Arrow Functions, and Async Execution
9. Combined Worked Example: Student JSON Toolkit
10. Quick Revision Checklist
11. Practice Exercises
12. Practice Answers


---

## Nullish Coalescing Operator ( `??` )

### The problem: fallbacks that destroy valid zeros

You often need a **fallback** when a value is missing: "if `x` has no real value, use `"hi"` instead." The older habit is the logical OR operator:

```
let y = x || "hi";
```

That works when missing means `null` / `undefined` / empty — but it also treats `0` and `false` as "missing." In production, a legitimate score of `0`, a quantity of `0`, or a boolean `false` silently gets replaced by the fallback. The lecture's warning in Hindi: *"Bahut saare bugs raise ho jaate hain in dono ke galat use ke chakkar mein"* — many bugs come from mixing up these two operators.

### Mechanism: only `null` and `undefined` trigger the right side

**Nullish coalescing** was introduced as a cleaner alternative to manual null/undefined checks.

```
let y = leftValue ?? fallbackValue;
```

| Left side | Result of `left ?? fallback` |
|---|---|
| `null` | `fallback` |
| `undefined` | `fallback` |
| `0` | `0` (kept) |
| `false` | `false` (kept) |
| `""` (empty string) | `""` (kept) |
| `"hello"` | `"hello"` |
| any other defined value | that value |

Rule in one sentence: **if the left side is `null` or `undefined`, return the right side; otherwise return the left side — even when the left side is falsy.**

### Lecture demonstration: `null`, then `0`, then `"hello"`

```
let x;
let y;

x = null;
y = x ?? "hi";
console.log(y);   // "hi"  — null → fallback

x = 0;
y = x ?? "hi";
console.log(y);   // 0     — zero is a valid value; NOT replaced

x = "hello";
y = x ?? "hi";
console.log(y);   // "hello"
```

### Direct comparison: `??` vs `||`

| Operator | Triggers fallback for | Keeps as-is |
|---|---|---|
| `\|\|` (OR) | `null`, `undefined`, `0`, `false` (and other falsy values) | only truthy values |

| Operator | Triggers fallback for | Keeps as-is |
|---|---|---|
| `??` (nullish) | `null`, `undefined` **only** | `0`, `false`, `""`, and every other defined value |

```
console.log(0 || "fallback");   // "fallback"  - BUG if 0 was legitimate
console.log(0 ?? "fallback");   // 0           - correct when 0 is valid
console.log(false || "x");      // "x"
console.log(false ?? "x");      // false
console.log(null ?? "x");       // "x"
console.log(undefined ?? "x");  // "x"
```

### Decision rule (exam-ready)

| Situation | Prefer |
|---|---|
| Only `null` / `undefined` should be excluded (scores, quantities, flags) | `??` |
| `0` and `false` should also count as empty / invalid | `\|\|` |

> **Exam:** `??` returns the right operand only when the left is `null` or `undefined`. `0 ?? "hi"` is `0`; `0 || "hi"` is `"hi"`.

> **Pitfall:** Using `||` for a score/quantity fallback. A real `0` becomes the fallback string/number and ships as a production bug.

---

## Arrays in JavaScript

### The problem classical definitions create

In C/C++/Java you are taught: an array is a **collection of similar data types**, with a **fixed size**, and an **index out of bounds** crash if you step past the end. That classical definition **does not apply** in JavaScript.

### Mechanism: dynamic, mixed-type collections

A JS array is a **collection of elements — similar or dissimilar**. There is **no type restriction**. Two key differences from C/C++/Java:

| Classical (C / Java) | JavaScript |
| --- | --- |
| Fixed size at declaration | **Dynamic** size — grows at runtime |
| Index out of bounds → error | **No** out-of-bounds error for adding; elements can be added freely |
| Same type required | Mixed types allowed |
| Closest cousin | Like Java's `ArrayList` / Collections |

### Declaration and access

Square brackets; mixed types are legal:

```
let arr = [10, 20, 30, "hello", true];
console.log(arr[0]);      // 10
console.log(arr[3]);      // "hello"
console.log(arr.length);  // 5
```

- Indexing starts at **0**
- Access any element via `arr[index]`
- `arr.length` returns the total element count
- In loops, use `i < arr.length` — **never hardcode** the last index

### Traversal with a classic `for` loop

```
let arr = [10, 20, 30, "hello", true];

for (let i = 0; i < arr.length; i++) {
    console.log(arr[i]);
}
```

Loop from `i = 0` to `arr.length - 1`; log `arr[i]` each iteration to print elements one by one.

### `const` arrays: reassignment vs mutation

```
const nums = [1, 2, 3];
nums.push(4);         // OK — mutates the array object
nums[0] = 99;         // OK — modifies an element
// nums = [9, 9, 9]; // TypeError — reassignment of the binding is forbidden
```

The lecture's phrase: *"Ye reassignment nahi hai, isliye possible hai"* — `push` / `pop` / element updates are **not** reassignment of the `const` variable; they mutate the array object the variable still

points to. So they remain valid.

> **Exam:** JS arrays are dynamic and untyped (mixed OK); size grows at runtime; use `arr.length` in loops. `const` blocks rebinding, not `push` / `pop` /index writes.

> **Pitfall:** Hardcoding `i < 5` instead of `i < arr.length` — breaks the moment the array grows or shrinks.

---

## 2D Arrays

### Correct definition for a developer

A **2D array is a collection of 1D arrays** — not "rows and columns." Rows/columns are a **visual analogy only**; technically incorrect for a developer who must reason about nested structures.

The same idea scales:

```
1D  = collection of values
2D  = collection of 1D arrays
3D  = collection of 2D arrays
4D  = collection of 3D arrays
 …
```

### Declaration and length

```
let arr = [
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
];

console.log(arr.length);    // 3 — number of INNER 1D arrays, not total elements
```

### Two-step indexing

| Expression | Result | Meaning |
|---|---|---|
| `arr[0]` | `[10, 20, 30]` | first inner 1D array |
| `arr[0][0]` | `10` | first element of first inner array |
| `arr[1][1]` | `50` | second element of second inner array |

You need **two index steps**: outer index picks which 1D array; inner index picks which element inside it.

### Nested-loop traversal

```
let arr = [
    [10, 20, 30],
    [40, 50],
    [70, 80, 90, 100]
];

for (let i = 0; i < arr.length; i++) {
    // arr[i] is itself a 1D array
    for (let j = 0; j < arr[i].length; j++) {
        console.log(arr[i][j]);
    }
}
```

- **Outer loop:** `i` from `0` to `arr.length` — each 1D array one by one
- **Inner loop:** `j` from `0` to `arr[i].length` — elements within that 1D array
- Inner arrays **do not need equal lengths**; `arr[i].length` handles jagged sizes automatically

```
arr ⟶ [  [10,20,30] ,  [40,50] ,  [70,80,90,100]  ]
            ↑              ↑              ↑
         arr[0]         arr[1]         arr[2]
         length 3       length 2       length 4
```

**Exam:** 2D = collection of 1D arrays. `arr.length` = number of inner arrays. Access with `arr[i][j]`. Inner loop bound is `arr[i].length` (supports jagged arrays).

**Pitfall:** Saying "2D array means rows and columns" in a technical answer — use the collection-of-1D-arrays definition.

---

## JSON (JavaScript Object Notation)

### The problem arrays alone cannot solve

Array elements are identified by **index only** — no semantic meaning. `arr[2]` does not tell a reader what that slot *means*. Structured records (student, product, order) need **named fields**.

### Mechanism: key–value objects

**JSON** = **JavaScript Object Notation**. Purpose: a structured way to store and represent data in **key–value pairs**.

```
let student = {
    id: 101,
    name: "Rohan",
    marks: 73,
    phone: "9876543210"
};

console.log(student.id);      // 101
console.log(student.name);    // "Rohan"
console.log(student.marks);   // 73
```

- Keys are **user-defined strings**
- Values can be any type: number, string, boolean, array, or **another JSON object**
- Access via **dot notation**: `student.id`, `student.name`, ...
- **Duplicate keys:** the later value **overwrites** the earlier one; both cannot coexist

| Approach | How you read a field | Self-documenting? |
|---|---|---|
| Array | `arr[2]` | No — index only |
| JSON | `student.marks` | Yes — key names carry meaning |

### Array as a value inside JSON

```
let student = {
    name: "Rohan",
    phone: ["9876543210", "9123456789"]
};

console.log(student.phone[0]);    // "9876543210"
```

### Nested JSON object as a value

```
let student = {
    name: "Rohan",
    address: {
        houseNo: "A56",
        street: "JS Nagar",
        state: "MH"
    }
```

```
};

console.log(student.address.houseNo);    // "A56"
```

### Array of JSON objects (server lists)

When a server returns **multiple records**, you store an **array of JSON objects**:

```
let studentsList = [
    { id: 101, name: "Rohan" },
    { id: 102, name: "Manoj" }
];

console.log(studentsList[0]);         // { id: 101, name: "Rohan" }
console.log(studentsList[0].name);    // "Rohan"
```

Iterate with a standard `for` loop and `studentsList[i].keyName`:

```
for (let i = 0; i < studentsList.length; i++) {
    console.log(studentsList[i].id, studentsList[i].name);
}
```

### Why JSON is universal for client–server exchange

JSON is the **universal standard** for client–server data exchange across technologies (Java, Python, PHP, .NET, JS). The **server sends JSON; the client sends JSON**; the format is the same regardless of the backend language.

*Background:* "JSON" here in class mostly means the **JS object literal / object notation** used to shape data. On the wire, that shape is usually serialised to a JSON **text** string ( `JSON.stringify` / `JSON.parse` appear in later networking work). Today's focus is the structured object shape itself.

> **Exam:** JSON = key–value structure; keys give meaning arrays lack. Nested objects via `obj.key.subKey`; lists of records via array of objects. Universal client–server format.

> **Pitfall:** Treating array indices as self-documenting for structured records — prefer named keys.

---

## Optional Chaining ( `?.` ) and Nested JSON Safety

### The crash pattern

- Accessing a key that **does not exist** on a JSON object returns `undefined` (no error): `student.xyz` → `undefined`
- Accessing a property **on** that `undefined` throws a **runtime error**: *"Cannot read properties of undefined"*

```
let student = { name: "Rohan" };
console.log(student.address);           // undefined – quiet
console.log(student.address.houseNo);   // TypeError – crash
```

### Real-world scenario from the lecture

A **backend bug** causes the `address` key to go missing from the server response. The frontend still runs `student.address.houseNo` → the application **crashes**. Incomplete or corrupted server data is normal in production; nested reads without protection are fragile.

### Mechanism: optional chaining

```
console.log(student?.address?.houseNo);
```

Before reading each key, `?.` checks whether the **current** value is defined. If `undefined` (or `null`) is encountered at any point, further reading **stops** and the expression returns `undefined` **silently**. No error; the application continues.

| Case | Expression | Result |
|---|---|---|
| `student` exists, `address` missing | `student?.address?.houseNo` | `undefined` (no crash) |
| `student` itself is `undefined` | `student?.address?.houseNo` | `undefined` (no crash) |
| Full path present | `student?.address?.houseNo` | the house number |

### Why not `??` for deep nesting?

For deeply nested objects with many levels, chaining `??` checks becomes **impractical**. Optional chaining is the **correct and concise** tool for safe nested reads. You can still combine them when you also want a fallback:

```
let house = student?.address?.houseNo ?? "N/A";
```

### Best practice

**Always** use optional chaining when reading nested JSON from a server — responses can be incomplete due to bugs or data corruption.

> **Exam:** Missing key → `undefined`. Property of `undefined` → crash. `a?.b?.c` short-circuits to `undefined` safely. Prefer `?.` for nested server JSON.

> **Pitfall:** Writing `student.address.houseNo` against live API data without `?.` — one missing key takes down the page.

---

## Destructuring and the Spread Operator

### The verbosity problem

In large applications, writing `product.name`, `product.price`, `product.pid` repeatedly across hundreds of places is **verbose and fragile**. If the backend renames a key, every occurrence must be updated manually.

### JSON destructuring

Extracts key–value pairs into individual `const` variables in one step:

```
const product = {
    pid: 1,
    name: "Laptop",
    price: 55000,
    quantity: 2,
    address: { houseNo: "A56", state: "MH" }
};

const { pid, name, price, quantity } = product;

console.log(pid, name, price);   // no product. prefix needed
```

- After destructuring, access `pid`, `name`, `price` directly — no object reference needed
- Uses `const` because server data is the **source of truth** and should not be mutated

### Nested object destructuring (lecture steps)

```
const { address } = product;
const { houseNo, state } = address;

console.log(houseNo, state);   // "A56" "MH"
```

Or in one nested pattern (same idea):

```
const { address: { houseNo, state } } = product;
```

### Array destructuring

Assigns elements to named variables **by position**:

```
const data = [10, 20, 30, 40];
const [x, y, z] = data;

console.log(x, y, z);   // 10 20 30  — data[0], data[1], data[2]
```

- If the array has **more** elements than named variables, extras are **ignored** ( `40` unused above)
- Stores a **copy at the time of destructuring**; later changes to the original array do **not** reflect in `x` , `y` , `z`
- The instructor noted this relates to **deep copy vs shallow copy** — recommended for independent exploration

### Spread operator ( `...` )

Spreads all key–value pairs (or array elements) of one value into another.

#### Merging two objects

```
const o1 = { a: 1, b: 2 };
const o2 = { c: 3, d: 4 };

const o3 = { ...o1, ...o2 };
console.log(o3);   // { a: 1, b: 2, c: 3, d: 4 } — flat merge

const wrong = { o1, o2 };
console.log(wrong); // { o1: {...}, o2: {...} } — nested keys, not intended
```

Without spread, `o1` and `o2` become **nested keys** inside the new object — not a flat merge.

### Merging two arrays

```
const arr1 = [1, 2];
const arr2 = [3, 4];

const combined = [ ... arr1,  ... arr2];
console.log(combined);    // [1, 2, 3, 4] – flat 1D

const nested = [arr1, arr2];
console.log(nested);      // [[1, 2], [3, 4]] – 2D, not a flat merge
```

| Goal | Without spread | With spread |
|---|---|---|
| Merge objects | Nested under key names | Flat key–value union |
| Merge arrays | 2D array of the two arrays | Flat 1D concatenation |

The lecture also noted that the `+` **operator does not work for merging objects**.

> **Exam:** `const { a, b } = obj` pulls named fields. `const [x, y] = arr` pulls by index. `{ ... o1,  ... o2}` and `[ ... a,  ... b]` flatten merges; without `...` you nest.

> **Pitfall:** Writing `const o3 = { o1, o2 }` or `const c = [arr1, arr2]` when you meant a flat merge — you get nesting instead.

---

## OOP in JavaScript: Classes, Constructors, and Inheritance

### The problem: objects without a blueprint

Day 3 treated "everything is an object." Day 4 adds **classes** — reusable blueprints — with Java-like syntax but several JS-specific rules that will trip anyone who copies Java habits blindly.

### Class and constructor rules

- Class declaration syntax is **the same shape as Java**
- Properties **cannot** be declared directly at class level the Java way ( `int pid;` at class body) in the style taught here
- Properties must be declared and **initialised inside the constructor** using `this`

```
class Student {
    constructor(id, name) {
        this.id = id;
        this.name = name;
    }

    display() {
        console.log(this.id, this.name);
    }
}

let s1 = new Student(101, "Rohan");
s1.display();
```

| Rule | Java | JavaScript (this lecture) |
|---|---|---|
| Constructor name | Class name | Always the keyword `constructor` |
| Inserting `this.` | Often compiler-aided | **Developer must write `this` manually** — no compiler auto-insert |
| Object creation | `new ClassName(args)` | Same — constructor runs automatically |
| Methods | May use various forms | **No `function` keyword** — write `methodName() {}` directly |
| Inside methods | `this.field` | Always `this.id`, `this.name` for instance properties |

- **Default constructor:** no parameters; hardcoded values inside
- **Parameterised constructor:** values passed at `new ClassName(args)`

## Encapsulation

There are **no `private` / `public` / `protected` access specifiers** in plain JS as taught here. **TypeScript** is required for that style of data hiding.

## Inheritance

```
class Person {
    constructor(name) {
        this.name = name;
    }
    greet() {
        console.log("Hi", this.name);
```

```
    }
}

class Student extends Person {
    constructor(name, marks) {
        super(name);          // MUST call before using this
        this.marks = marks;
    }
    greet() {
        console.log("Student", this.name, this.marks);
    }
}

let s = new Student("Rohan", 73);
s.greet();   // overriding in action
```

- Syntax: `class B extends A`
- Child gets all properties and methods of parent automatically
- If the child defines its **own** constructor, `super()` **must be called manually** before using `this`
- Lecture contrast: *"Java mein compiler `super()` call karta tha; JS mein developer ko khud karna padta hai"*

## Types of inheritance in JS

| Type | Supported? | Notes |
|---|---|---|
| Single | Yes | One parent |
| Multi-level ( `A → B → C → D` ) | Yes | Chain of extends |
| Hierarchical (one parent, many children) | Yes | Multiple classes extend same parent |
| Multiple inheritance | **Not** natively | Workaround via **mixins**; TypeScript supports related ideas via **interfaces** |

## Polymorphism

| Feature | In JS? |
|---|---|
| Function / method **overriding** | **Supported** — redefine a parent method in the child |
| Function **overloading** | **Not supported** (needs a compiler / static types) |

> **Exam:** Properties via `this` in `constructor`. Constructor is always named `constructor`. Child with own constructor must `super()` first. Overriding yes; overloading no. No classical access modifiers without TypeScript.

> **Pitfall:** Naming the constructor after the class (Java habit), forgetting `this.`, or omitting `super()` in a child constructor.

---

## Callback Functions, Arrow Functions, and Async Execution

### Functions as first-class objects

In JS, functions are **first-class objects**: they can be assigned to variables, **passed as arguments**, and returned from other functions. That single idea unlocks callbacks, array methods, and async APIs.

### Callback function

A **callback** is a function passed as an argument to another function. The **outer** function calls the callback internally.

```
function demo() {
    console.log("I am the callback");
}

function show(fn) {
    console.log("show is running");
    fn();   // call the callback
}

show(demo);   // demo is the callback
```

### Anonymous function

A function defined **inline without a name** — used when it only needs to exist as an argument and will never be called from outside.

Lecture phrase: *"Bahar ki duniya mein koi kaam nahi, toh naam dene ka fayda nahi"* — if the outside world never needs to call it by name, naming it has no benefit.

```
show(function () {
    console.log("anonymous callback");
```

```
});
```

## Arrow function syntax

A newer, concise way to define functions — equivalent in spirit to **Java's lambda expressions**.

```
// Standard
const add = (x, y) => {
    return x + y;
};

// Single return: omit return + curly braces
const add2 = (x, y) => x + y;

// Single argument: omit parentheses
const plusFive = n => n + 5;
```

| Shortcut | When allowed | Example |
|---|---|---|
| Omit `return` and `{ }` | Single expression return | `(x, y) => x + y` |
| Omit `( )` around params | Exactly one parameter | `n => n + 5` |

## Array methods that take callbacks

### `forEach` — visit every element

Receives `(value, index)`:

```
let arr = [10, 20, 30];
arr.forEach((value, index) => console.log(value, index));
```

### `filter` — keep elements where callback returns `true`

```
let arr = [1, 2, 3, 4, 5, 6];
const evens = arr.filter(value => value % 2 === 0);
console.log(evens);   // [2, 4, 6]
```

The expression `value % 2 === 0` is **already a boolean** — no need for an explicit `if` / `return true` / `return false` when you use the concise arrow form.

### `map` — build a **new** array from returned values

```
const doubled = arr.map(value => value * 2);
console.log(doubled);   // new array
```

```
console.log(arr);       // original unchanged
```

`reduce`

Flagged for **independent exploration** (not expanded in class).

| Method | Callback job | Result |
|---|---|---|
| `forEach` | Side effect per element | `undefined` (no new array) |
| `filter` | Return boolean keep/drop | New array of kept elements |
| `map` | Return transformed value | New array of same length |
| `reduce` | (self-study) | Usually one accumulated value |

## Synchronous vs asynchronous execution

| Mode | Behaviour | When required |
|---|---|---|
| **Synchronous** (default) | Every process **blocks** the next; strict sequence | Tasks **depend** on each other's output (e.g. sum → factorial of sum → square of that) |
| **Asynchronous** | **Non-blocking**; tasks overlap; results arrive in **order of completion**, not invocation | Tasks are **independent** of each other |

## `setTimeout(callback, ms)` — simulated async delay

Built-in async function: starts a timer, does **not** block the rest of the program, calls the callback once the timer expires.

Lecture demo — three independent operations:

| Task | Delay | "Started" logs | Result order |
|---|---|---|---|
| Sum | 1500 ms | Immediate | 1st (at 1.5 s) |
| Factorial | 3000 ms | Immediate | 3rd (at 3 s) |
| Square | 2000 ms | Immediate | 2nd (at 2 s) |

All three *"started calculating..."* messages print **immediately**. Results arrive: **sum** → **square** → **factorial**. Order of results **differs** from order of invocation — proof of async behaviour.

Lecture phrase: *"Teen timers ek saath chal rahe hain; jiska timer pehle complete hoga, uska result pehle milega"* — three timers run together; whichever finishes first delivers its result first.

```
console.log("sum started");
setTimeout(() => {
    console.log("sum result", 10 + 20);
}, 1500);

console.log("factorial started");
setTimeout(() => {
    let n = 5, f = 1;
    for (let i = 1; i ≤ n; i++) f *= i;
    console.log("factorial result", f);
}, 3000);

console.log("square started");
setTimeout(() => {
    console.log("square result", 12 * 12);
}, 2000);

// Console order of RESULTS (approx): sum @1.5s, square @2s, factorial @3s
```

**Exam:** Callback = function passed as argument. Arrow shortcuts: one expr → no `return` / `{}`; one arg → no `()`. `filter` keeps `true`; `map` builds new array, original intact. Async results follow completion order, not call order.

**Pitfall:** Expecting `setTimeout` tasks to finish in the same order you scheduled them when delays differ.

---

## Combined Worked Example: Student JSON Toolkit

One HTML file that ties together the day's practical topics: a student JSON array, optional chaining, nullish coalescing, destructuring, array callbacks, a tiny class, and the three assignment tasks (search by id, topper, failed list).

```
<!DOCTYPE html>
<html>
<head>
    <title>WPT Day 4 – Student Toolkit</title>
</head>
<body>
    <h1>Student JSON Toolkit</h1>
```

```
    <p>Open the browser console (Inspect → Console) for detailed logs.</p>

    <script>
        // --- Source data: array of JSON objects (server-shaped) ---
        const studentsList = [
            { id: 101, name: "Rohan",  marks: 73, address: { houseNo: "A56", state:
"MH" } },
            { id: 102, name: "Manoj",  marks: 28 },
            { id: 103, name: "Priya",  marks: 91, address: { houseNo: "B12", state:
"KA" } },
            { id: 104, name: "Asha",   marks: 33 },
            { id: 105, name: "Vikram", marks: 0,  address: { houseNo: "C7",  state:
"MH" } }
            // note: some records omit address — simulates incomplete server data
        ];

        // --- 1. Search by ID (optional chaining on .id) ---
        const searchId = 103;   // hardcoded "user input" as in the assignment

        function findById(list, id) {
            for (let i = 0; i < list.length; i++) {
                if (list[i]?.id === id) {
                    return list[i];
                }
            }
            return null;
        }

        const found = findById(studentsList, searchId);
        console.log("Search by id", searchId, "→", found);

        // --- 2. Find the topper (compare object.marks, not bare numbers) ---
        function findTopper(list) {
            let topper = list[0];
            for (let i = 1; i < list.length; i++) {
                const currentMarks = list[i]?.marks ?? -1;
                const topMarks = topper?.marks ?? -1;
                if (currentMarks > topMarks) {
                    topper = list[i];
                }
            }
            return topper;
        }

        const topper = findTopper(studentsList);
        // Destructuring: pull named fields once instead of repeating topper.name etc.
        const { name: topName, marks: topMarks } = topper;
        console.log(`Topper: ${topName} with ${topMarks}`);

        // --- 3. Failed students: marks ≤ 33 ---
        function findFailed(list) {
```

```
            const failed = [];
            for (let i = 0; i < list.length; i++) {
                const m = list[i]?.marks;
                if (m !== undefined && m <= 33) {
                    failed.push(list[i]);
                }
            }
            return failed;
        }

        const failed = findFailed(studentsList);
        console.log("Failed students:", failed);

        // Same filter idea with an arrow callback (Day 4 array method)
        const failedViaFilter = studentsList.filter(s => (s?.marks ?? 100) <= 33);
        console.log("Failed via filter:", failedViaFilter);

        // map: build display strings; original array unchanged
        const labels = studentsList.map(s => {
            const marks = s?.marks ?? "N/A";
            return `${s?.name ?? "Unknown"}: ${marks}`;
        });
        console.log("Labels:", labels);

        // ?? vs || : Vikram has marks 0 — a legitimate score
        const vikram = studentsList.find(s => s?.id === 105);
        console.log("marks || 'absent' →", vikram.marks || "absent"); // wrongly
"absent"
        console.log("marks ?? 'absent' →", vikram.marks ?? "absent"); // correctly 0

        // Nested JSON safety
        studentsList.forEach((s, index) => {
            const house = s?.address?.houseNo ?? "no address on record";
            console.log(`[${index}] ${s?.name} → ${house}`);
        });

        // Spread: merge two partial records into one flat object
        const patch = { phone: "9876543210" };
        const enriched = { ...found, ...patch };
        console.log("Enriched found student:", enriched);

        // Tiny class + inheritance flavour from the OOP segment
        class Person {
            constructor(name) {
                this.name = name;
            }
            describe() {
                return `Person: ${this.name}`;
            }
        }
```

```
        class ExamStudent extends Person {
            constructor(name, marks) {
                super(name);
                this.marks = marks;
            }
            describe() {
                // overriding
                return `ExamStudent: ${this.name} scored ${this.marks}`;
            }
        }

        const es = new ExamStudent(topName, topMarks);
        console.log(es.describe());

        // Async demo: independent timers (order of results ≠ order of schedule)
        console.log("async: scheduling three independent jobs…");
        setTimeout(() => console.log("async result: sum", 10 + 20), 1500);
        setTimeout(() => {
            let n = 5, f = 1;
            for (let i = 1; i ≤ n; i++) f *= i;
            console.log("async result: factorial", f);
        }, 3000);
        setTimeout(() => console.log("async result: square", 12 * 12), 2000);
    </script>
</body>
</html>
```

### Trace it end to end

1. **Data shape.** `studentsList` is an **array of JSON objects** — the same shape a server would send. Some objects omit `address` on purpose (backend-bug simulation).
2. **Search by id.** Loop compares `list[i]?.id` so a missing `id` key does not crash; returns the matching object or `null`.
3. **Topper.** Unlike a plain numeric array, you extract the **object**, then compare `object.marks`. `?? -1` guards missing marks during comparison.
4. **Failed list.** Collect where `marks ≤ 33` (includes exactly 33 and Vikram's `0`). The `filter` version is the same rule expressed as an arrow callback.
5. `??` **vs** `||`**.** Vikram's `0` must stay `0`. `||` wrongly substitutes `"absent"`; `??` keeps `0`.
6. **Optional chaining.** `s?.address?.houseNo ?? "no address…"` never throws when `address` is missing.
7. **Destructuring & spread.** Topper fields become `topName` / `topMarks`; `{ ...found, ...patch }` flat-merges a phone field.
8. **Class / override.** `ExamStudent` calls `super(name)`, then overrides `describe()`.

9. **Async.** Three `setTimeout` calls schedule immediately; results print **sum** → **square** → **factorial** (1.5s / 2s / 3s), not schedule order.

If you can explain those nine steps without scrolling up, Day 4's practical thread is under control.

---

## Quick Revision Checklist

| Topic | One-line takeaway |
|---|---|
| `??` | Fallback only for `null` / `undefined`; keeps `0` and `false` |
| `\|\|` vs `??` | `\|\|` also treats `0` / `false` as empty — common production bug |
| JS array | Mixed types OK; dynamic size; like ArrayList; no classical "same type" rule |
| `arr.length` | Use in loop bounds; never hardcode the last index |
| `const` array | Rebind forbidden; `push` / `pop` /index writes still OK |
| 2D array | Collection of 1D arrays — not "rows and columns" as a technical definition |
| `arr[i][j]` | Outer picks inner array; inner picks element; use `arr[i].length` |
| JSON | Key–value structure; keys give meaning arrays lack |
| Nested JSON | `obj.a.b`; array of objects for server lists |
| Client–server | JSON is the universal exchange format across languages |
| Missing key | Returns `undefined` (quiet) |
| Prop of `undefined` | Runtime crash — "Cannot read properties of undefined" |
| `?.` | Safe nested read; short-circuits to `undefined` |
| Destructure object | `const { a, b } = obj` — prefer `const` for server data |
| Destructure array | `const [x, y] = arr` — by position; extras ignored |
| Spread object | `{ ...o1, ...o2 }` flat merge; without `...` → nested keys |
| Spread array | `[...a, ...b]` flat 1D; without `...` → 2D |
| Class props | Init with `this` inside `constructor` (always named `constructor`) |
| `super()` | Child constructor must call it manually before `this` |

| Topic | One-line takeaway |
|---|---|
| Inheritance types | Single, multi-level, hierarchical yes; multiple no (mixins / TS) |
| Polymorphism | Overriding yes; overloading no |
| Encapsulation | No private/public/protected in plain JS — use TypeScript |
| Callback | Function passed as an argument; outer calls it |
| Anonymous fn | Inline, unnamed — no outside reuse → no need for a name |
| Arrow | `(x,y)⇒x+y` ; one arg → `n⇒…` ; Java-lambda cousin |
| `forEach` | Callback per element with `(value, index)` |
| `filter` | Keep where callback is `true` ; expr can be the boolean itself |
| `map` | New array of returned values; original unchanged |
| Sync vs async | Dependent tasks → sync; independent → async |
| `setTimeout` | Non-blocking timer; results by completion order, not schedule order |

---

## Practice Exercises

Work these without looking at the answers. They target the exact confusions the lecture called out.

### Concepts — `??` , arrays, 2D, JSON

1. Predict `y` in each case: (a) `y = null ?? "hi"` (b) `y = 0 ?? "hi"` (c) `y = 0 || "hi"` . Which pair is the production bug the lecture warned about?
2. Why does the classical "collection of similar data types / fixed size" definition fail for JavaScript arrays? Name the Java collection the lecture compared them to.
3. True or false, with justification: "A `const` array cannot use `push` ." Quote the lecture's reasoning in your own words.
4. Give the developer-correct definition of a 2D array. What does `arr.length` return for `[[10,20,30],[40,50,60],[70,80,90]]` ?
5. Why prefer JSON over a plain array for a student record with id, name, and marks?

### Optional chaining, destructuring, spread

6. A server omits `address`. What does `student.address` return? What happens on `student.address.houseNo`? How do you write the safe read?
7. Why is optional chaining preferred over a long chain of `??` for deep nested JSON?
8. Write one line that destructures `pid`, `name`, and `price` from `product` into `const` bindings.
9. Predict the shape: (a) `const o3 = { o1, o2 }` (b) `const o3 = { ...o1, ...o2 }` (c) `const c = [arr1, arr2]` (d) `const c = [...arr1, ...arr2]`.
10. After `const [x, y, z] = [10, 20, 30, 40]`, what are `x`, `y`, `z`? What happens to `40`?

### OOP and callbacks / async

11. In JS (this lecture), what must the constructor method be named? Where are instance properties initialised, and with which keyword?
12. A child class defines its own constructor and uses `this` before `super()`. What goes wrong, and how does Java differ according to the lecture?
13. Which inheritance types are supported natively? How do you get a multiple-inheritance-like effect?
14. Overriding vs overloading in JS — which works, which does not, and why?
15. In one sentence each, define: callback function; anonymous function. What Hindi rationale did the lecture give for anonymous functions?
16. Rewrite `function(n) { return n + 5; }` as the shortest legal arrow form.
17. What boolean expression can you pass to `filter` to keep even numbers, and why is an `if` unnecessary in the concise form?
18. Three `setTimeout` calls are scheduled for 1500 ms (sum), 3000 ms (factorial), and 2000 ms (square). In what order do the **results** arrive? What phrase captured the rule?

### Assignment-style

19. On an array of student objects, explain the key difference between finding a max in a **numeric** array vs finding the **topper**.
20. Spot the bug: `let displayMarks = student.marks || "N/A";` when `marks` can legitimately be `0`. Fix it.

---

## Practice Answers

1. (a) `"hi"`. (b) `0`. (c) `"hi"`. The bug pair is **(b) vs (c)** — using `||` when `0` is valid replaces a real score/quantity with the fallback.
2. JS arrays allow **mixed types**, grow **dynamically** at runtime, and do not throw classical out-of-bounds errors when you add freely. Compared to Java's `ArrayList` / Collections.
3. **False.** `const` forbids **reassignment** of the binding (`arr = …`), not mutation. `push` / `pop` /index writes are allowed because *"ye reassignment nahi hai."*
4. A 2D array is a **collection of 1D arrays**. `arr.length` is `3` (count of inner arrays), not the total number of numbers.
5. Array slots are index-only (`arr[2]` has no meaning). JSON keys are self-documenting (`student.marks`).
6. `student.address` → `undefined`. `student.address.houseNo` → **runtime error** ("Cannot read properties of undefined"). Safe: `student?.address?.houseNo`.
7. Deep nesting makes a ladder of `??` checks **impractical**; `?.` is the concise tool that stops at the first missing link.
8. **One-line destructuring:**

```
const { pid, name, price } = product;
```

9. (a) **Nested** `{ o1: {…}, o2: {…} }`. (b) **Flat** merge of keys. (c) **2D** `[[…],[…]]`. (d) **Flat** 1D concatenation.
10. `x=10`, `y=20`, `z=30`. The extra `40` is **ignored**.
11. Always named `constructor`. Properties are initialised **inside the constructor** with `this` (e.g. `this.id = id`).
12. Using `this` before `super()` fails — `super()` **must be called first**. In Java the compiler often inserted `super()`; in JS the **developer** must write it.
13. **Single, multi-level, hierarchical** — yes. **Multiple** — not native; workaround via **mixins**; TypeScript offers related support via **interfaces**.
14. **Overriding** supported (redefine method in child). **Overloading** not supported — needs a compiler / static resolution.
15. **Callback:** function passed as an argument to another function (which calls it). **Anonymous:** inline function with no name, only used as an argument. Rationale: if the outside world has no use for it, naming has no benefit (*"bahar ki duniya mein koi kaam nahi…"*).

16. **Shortest arrow:**

```
n ⇒ n + 5
```

17. `value ⇒ value % 2 === 0` — the `%` comparison **is already a boolean**, so the concise arrow can return it directly without `if` / `return true` / `return false`.
18. Results: **sum (1.5s), then square (2s), then factorial (3s)**. Rule: three timers run together; **whichever timer completes first delivers its result first**.
19. In a numeric array you compare numbers directly. For a topper you must first take each **object**, then compare `object.marks` (and typically keep the whole student object as the winner).
20. Bug: `||` treats `0` as empty → shows `"N/A"`. Fix:

```
let displayMarks = student.marks ?? "N/A";
```